



## TÉCNICO SOLAR BOAT

UNIVERSIDADE DE LISBOA - INSTITUTO SUPERIOR TÉCNICO

---

# Autonomous Systems Recruitment

## Phase 1 Report

---

**Author:** Guilherme Duarte Esteves  
Last Change: March 20, 2026



# Index

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>1</b>  |
| 1.1      | Objective . . . . .                                       | 1         |
| <b>2</b> | <b>ROS 2 - Robot Operating System 2</b>                   | <b>2</b>  |
| 2.1      | The Evolution: ROS 1 vs. ROS 2 . . . . .                  | 2         |
| 2.2      | Core Concepts and Communication Primitives . . . . .      | 3         |
| 2.3      | System Configuration and Tooling . . . . .                | 4         |
| 2.4      | Applications and Benefits for the SP01 and TSB . . . . .  | 4         |
| <b>3</b> | <b>Fossen's Marine Craft Model</b>                        | <b>5</b>  |
| 3.1      | Kinematics and Reference Frames . . . . .                 | 5         |
| 3.2      | The 3-DOF Simplification for Surface Vessels . . . . .    | 6         |
| 3.3      | Kinetics: The Equation of Motion . . . . .                | 7         |
| <b>4</b> | <b>Unreal Engine</b>                                      | <b>8</b>  |
| 4.1      | Photorealism for Perception and Computer Vision . . . . . | 8         |
| 4.2      | Fluid Dynamics and the Chaos Physics Engine . . . . .     | 9         |
| 4.3      | SITL and Fossen's Integration . . . . .                   | 9         |
| <b>5</b> | <b>Conclusion</b>                                         | <b>10</b> |
|          | <b>References</b>                                         | <b>11</b> |

# 1 Introduction

The development of Autonomous Surface Vehicles presents a unique set of multidisciplinary engineering challenges. Unlike ground robots, marine prototypes must navigate harsh, unpredictable, and highly dynamic environments where wind, waves, and water resistance constantly act upon the hull. For the Técnico Solar Boat team, successfully deploying a vessel like the SP01 requires a software architecture that is not only highly intelligent but also deeply integrated with the physical realities of the ocean.

To achieve full autonomy, a modern robotic vessel relies on a synergy of robust software middleware, precise mathematical modeling, and high-fidelity simulation. This report explores the three foundational pillars that enable this unity for an autonomous vessel: the Robot Operating System (ROS 2) for decentralized system communication, Fossen's Marine Craft Model for accurate dynamic control, and Unreal Engine for safe, realistic Software-in-the-Loop (SITL) testing. Together, these technologies form a cohesive pipeline that bridges the gap between digital algorithms and physical maritime navigation.

## 1.1 Objective

The primary objective of this report is to research, synthesize, and evaluate the core technologies required by the Autonomous Systems department to build a robust, navigation ready stack for an autonomous vessel.

Specifically, this document aims to:

- Detail the functioning of ROS 2, highlighting how its decentralized architecture and real-time capabilities eliminate single points of failure.
- Explain the application of Fossen's mathematical model, particularly the 3-DOF simplification, to predict the vessel's behavior and counteract hydrodynamic drag.
- Demonstrate how Unreal Engine can be utilized as a SITL simulation environment to safely train perception AI and tune control algorithms before physical deployment.

Ultimately, this report serves as a strategic roadmap, illustrating how the seamless integration of the domains of software, physics, and simulation will maximize the safety, reliability, and competitive performance of a marine craft developed by the TSB's team.

## 2 ROS 2 - Robot Operating System 2

The *Robot Operating System* (ROS 2) is an open-source middleware suite that has become the industry standard for robotics software development. Despite its name, ROS 2 is not a traditional operating system but rather a comprehensive software framework. It provides essential services designed for heterogeneous computer clusters, including hardware abstraction, low-level device control, and robust inter-process message passing.

### 2.1 The Evolution: ROS 1 vs. ROS 2

While the original ROS 1 revolutionized robotics research, it was built around a centralized architecture relying on a single “Master” node (`roscore`). If this master node crashed or lost connection, the entire communication network collapsed, creating a critical single point of failure. Furthermore, ROS 1 utilized custom communication protocols that lacked true real-time guarantees.

ROS 2 was engineered from the ground up to solve these industrial limitations. By replacing the custom protocols with the *Data Distribution Service* (DDS) standard, ROS 2 eliminates the central master node, enabling peer-to-peer discovery between all components. This transition brings true real-time execution capabilities, introduces granular *Quality of Service* (QoS) network policies, and completely removes the single point of failure, upgrades that are absolutely mandatory for deploying autonomous vehicles in harsh, unpredictable environments like the open sea. [1, 2]



Robot Operating System

## 2.2 Core Concepts and Communication Primitives

To fully leverage its distributed nature, the ROS 2 architecture is broken down into several fundamental components and communication paradigms [1, 2]:

- **Nodes:** The base computational unit in ROS 2 is the node. Nodes act as independent processes responsible for handling specific devices or executing computing algorithms, for example: one node dedicated to reading LiDAR data, and another to path planning. This modularity ensures that a software crash in a single sensor node does not bring down the entire vessel’s system.
- **Topics:** Topics are asynchronous data streams used to exchange information between nodes using a *Publish-Subscribe* pattern. They are designed for the continuous, high-frequency transmission of a single message type, such as sensor readouts or target motor speeds. While a node typically acts as either a publisher or a subscriber for a specific task, the architecture imposes no restrictions on the number of concurrent publishers and subscribers connected to a single topic.
- **Services:** Unlike topics, communication through services implements a synchronous, client-server model. In this paradigm, a server node registers a service, and client nodes can send a request and wait for a response. Because a request can also contain payload data, services allow for structured two-way communication. They are ideal for quick, discrete operations, such as querying the current state of a subsystem or switching a sensor’s operational mode.
- **Actions:** For tasks that are time-consuming or require execution tracking, ROS 2 utilizes Actions. Built on top of topics and services, actions provide a preemptable client-server model. They allow a node to send a “goal” (e.g., “Maps to waypoint X”), receive continuous feedback during execution, and ultimately obtain a final result, with the crucial ability to cancel the request mid-execution if the environment changes.

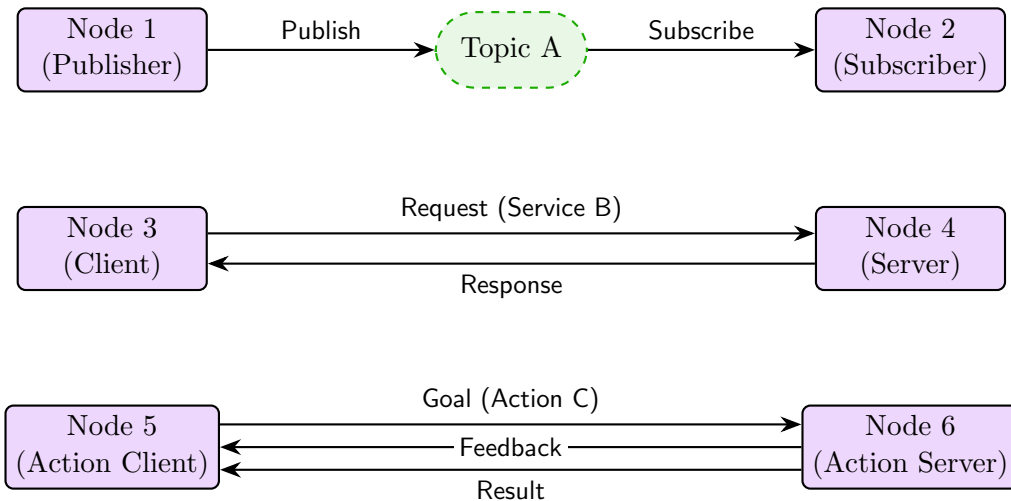


Figure 1: Conceptual diagram of ROS 2 Communication Primitives.

## 2.3 System Configuration and Tooling

Beyond real-time communication, ROS 2 provides robust tooling for system configuration, deployment, and data analysis [1, 2, 3]:

- **Node Parameters:** Parameters act as the configuration interface for a node. They consist of key-value pairs (such as integers, floats, or booleans) that can be loaded at startup or dynamically modified at runtime. This allows engineers to tune algorithm variables, like the proportional gain of a PID (Proportional-Integral-Derivative) controller or the detection threshold of a camera, without needing to recompile the source code.
- **Bag Files:** In autonomous systems development, `rosbag2` is an indispensable tool. It allows developers to serialize and record the exact message data flowing through topics during a live operation into a “bag file.” These files can later be played back in a simulated environment, injecting the exact same sensor data into the computation graph. This is crucial for safely debugging perception and control algorithms offline.
- **Packages and Workspaces:** ROS 2 software is strictly organized into packages. A single package generally encapsulates the code, parameters, and build configurations required to perform a specific type of task. These packages are grouped into “workspaces” and compiled using build tools like `colcon`, ensuring that the codebase remains maintainable, reusable, and easy to deploy across the team’s heterogeneous hardware cluster.

## 2.4 Applications and Benefits for the SP01 and TSB

For the TSB team, adopting ROS 2 is not merely a software choice, it is a strategic engineering enabler for the SP01 prototype. The highly modular nature of ROS 2 allows the Autonomous Systems department to divide complex challenges, such as computer vision, mapping, and motor actuation, into isolated nodes. This means multiple sub-teams can develop, test, and compile their algorithms concurrently without causing merge conflicts or breaking the entire system.

Furthermore, the hardware abstraction provided by ROS 2 means the exact same navigation code can be tested in a virtual physics simulation and seamlessly deployed to the physical vessel without major modification. Ultimately, the decentralized fault tolerance of ROS 2 ensures that if a non-critical component fails during a competition, the core navigation and control nodes will remain fully operational, thereby maximizing the prototype’s safety, reliability, and competitive performance.

### 3 Fossen’s Marine Craft Model

In the development of Autonomous Surface Vehicles like the SP01, the software architecture must bridge the gap between digital navigation commands and physical reality. The Marine Craft Model created by Thor I. Fossen provides the industry standard mathematical framework for expressing the highly non-linear dynamics of marine vessels. For the autonomous systems department, understanding this model is critical. In order to design accurate trajectory planners and robust PID controllers, the software must be able to predict how the vessel’s physical mass will react to motor thrust while battling the resistance of the water [4, 5].

#### 3.1 Kinematics and Reference Frames

The foundation of Fossen’s model relies on decoupling the vehicle’s position from its velocity using two distinct coordinate systems. The position and orientation of the boat are measured in the Earth-fixed frame, often referred to as the North-East-Down (NED) frame. This is the global map where GPS coordinates live. However, the velocities and the forces generated by the thrusters are measured in the Body-fixed frame, a coordinate system that moves and rotates with the boat itself. To translate the boat’s local velocity into global map movement, the model uses a kinematic transformation matrix to bridge the two frames [4, 5].

### 3.2 The 3-DOF Simplification for Surface Vessels

A fully unconstrained underwater vehicle (like a submarine) moves in six Degrees of Freedom (6-DOF): Surge, Sway, Heave, Roll, Pitch, and Yaw. However, modeling 6-DOF requires complex  $6 \times 6$  matrices that consume significant computational resources. For surface vessels, like the SP01, the buoyancy and hull design passively stabilize the vertical and rotational axes. Therefore, typically the control algorithms can safely reduce the model to 3 Degrees of Freedom (3-DOF), focusing exclusively on [4, 5, 6]:

- **Surge:** Forward and backward translation.
- **Sway:** Lateral translation (drifting sideways).
- **Yaw:** Rotation around the vertical axis (steering left or right).

This 3-DOF simplification drastically reduces the mathematical complexity, allowing the onboard ROS 2 nodes to compute control algorithms much faster.

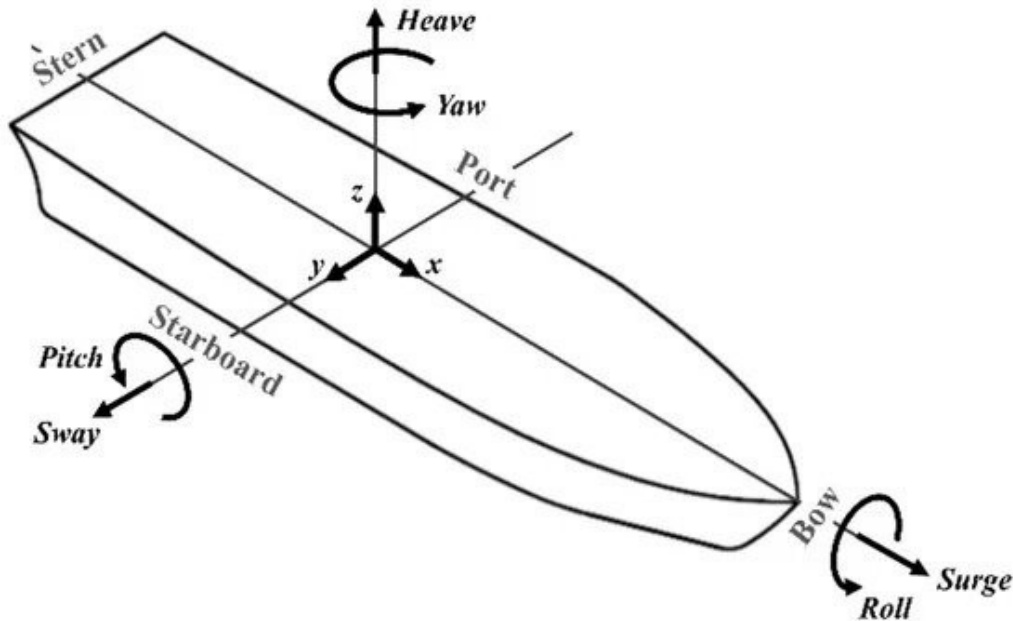


Figure 2: Representation of the 6 Degrees of Freedom of a marine vessel [7].

### 3.3 Kinetics: The Equation of Motion

The core of Fossen’s framework is the vectorial equation of motion, which calculates the exact force required to move the boat. It is defined as:

$$M\dot{\nu} + C(\nu)\nu + D(\nu)\nu + g(\eta) = \tau$$

For a control systems engineer, each term in this equation represents a specific physical hurdle that the software must overcome:

- **$M$  (Inertia and Added Mass):** This matrix accounts not only for the physical weight of the SP01, but also the “added mass” of the water surrounding the hull that must be pushed out of the way when the boat accelerates.
- **$C$  (Coriolis and Centripetal Forces):** This matrix calculates the forces induced by the rotation of the boat, ensuring the software anticipates how the vessel will slide sideways during a sharp turn.
- **$D$  (Hydrodynamic Damping):** This is the friction and resistance of the water. Unlike air resistance, water drag is highly non-linear. As the SP01 accelerates, the damping forces increase exponentially, making this the most challenging variable for the control algorithms to manage.
- **$g$  (Hydrostatic Forces):** These are the gravitational and buoyancy forces, which are mostly negligible in a simplified 3-DOF surface model.
- **$\tau$  (Control Inputs):** This represents the actual thrust and torque generated by the SP01’s propulsion system to overcome all the forces listed above.

By implementing Fossen’s model within the software stack, often within a simulation environment like Unreal Engine, the autonomous systems team can safely test their navigation algorithms against realistic water physics before ever placing the SP01 in the water [4].

## 4 Unreal Engine

Before a highly valuable prototype like the SP01 is deployed in the unpredictable open sea, its autonomous systems must be rigorously tested in a safe, controlled environment. This is achieved through *Software-in-the-Loop* (SITL) simulation. While older robotic simulators like Gazebo excel at basic kinematic testing, modern autonomous marine development is rapidly shifting toward Unreal Engine 5 (UE5). By leveraging UE5’s advanced rendering and physics engines, the Autonomous Systems team can create a digital twin of the boat and the ocean, allowing for the extensive testing of perception, navigation, and control algorithms [8, 9, 10].

### 4.1 Photorealism for Perception and Computer Vision

For an autonomous surface vehicle, the ocean is a visually complex environment. Water causes dynamic reflections, sun glare, and constantly shifting horizons, which can easily confuse computer vision algorithms. Unreal Engine’s state-of-the-art rendering system, Lumen, provides true photorealism.

By simulating realistic lighting, fog, rain, and varying times of day, engineers can generate highly accurate synthetic datasets. This allows the team to train and validate object detection models (such as YOLO networks used for buoy and obstacle recognition) in extreme weather conditions that would be dangerous or difficult to replicate in the real world. If the AI can reliably detect a virtual buoy reflecting sunlight in a UE5 storm, it is much more likely to succeed when that happens in reality [8, 9, 10].

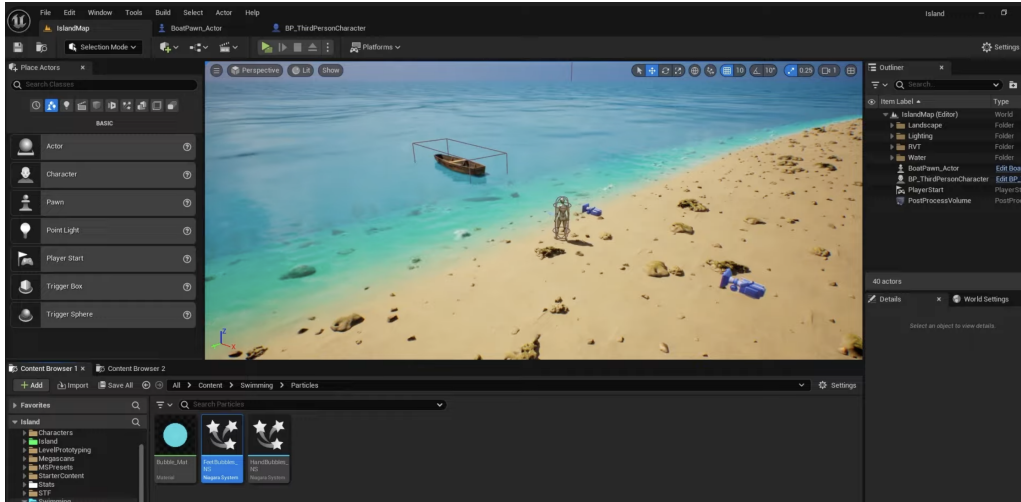


Figure 3: Example of Unreal Engine’s photorealistic rendering capabilities [10].

## 4.2 Fluid Dynamics and the Chaos Physics Engine

Beyond graphics, a marine simulator must replicate the physical behavior of water. Unreal Engine utilizes the Chaos physics engine combined with its dedicated Water Plugin to simulate dynamic fluid surfaces. The engine procedurally generates Gerstner waves, calculating the exact height and angle of the water at any given coordinate.

By attaching a “Buoyancy Component” to the 3D mesh of the SP01, the engine calculates the interactions between the hull’s shape, its center of mass, and the water surface. This provides precise simulation for the restorative hydrostatic forces (Heave, Pitch, and Roll) that were previously noted to be passively stabilized by the boat’s design [8, 9, 10].

## 4.3 SITL and Fossen’s Integration

Because Unreal Engine is originally a game engine, its default physics are optimized for visual performance rather than strict engineering accuracy. To transform UE5 into a true marine simulator, the software architecture must be extended in two critical ways:

- **Hydrodynamic Injection:** The default water physics lack the complex, non-linear drag of the real ocean. Engineers must use C++ to inject Fossen’s Marine Craft equations directly into the engine’s physics tick. By applying the Hydrodynamic Damping and Coriolis matrices as custom forces to the virtual hull, the digital boat mimics the exact inertia and drift of the real vessel.
- **ROS 2 Middleware Bridge:** The simulator must communicate with the boat’s actual software stack. Using integration plugins (such as `rc1UE`), Unreal Engine connects directly to the ROS 2 network. The simulation acts as a virtual sensor suite, publishing synthetic camera frames and LiDAR point clouds to ROS 2 topics. The navigation nodes process this data, make decisions, and publish motor commands back to Unreal Engine. The virtual motors apply thrust, the physics engine updates the boat’s position, and the loop continues.

Ultimately, mastering Unreal Engine allows the engineering team to iterate on control algorithms thousands of times in a virtual ocean, ensuring that the code deployed to the physical vessel is robust, safe, and ready [8, 9, 10].

## 5 Conclusion

Developing a fully autonomous marine vessel like the SP01 demands a robust coordination between software, physics, and simulation. As explored in this report, the integration of ROS 2, Fossen’s Marine Craft Model, and Unreal Engine provides the Técnico Solar Boat team with an advanced engineering pipeline capable of overcoming the unique challenges that the maritime environment imposes.

Each technology plays a distinct, crucial role in the vessel’s autonomy stack. ROS 2 serves as the decentralized nervous system, ensuring that sensor data and motor commands flow in real time without the risk of a single point of failure. Fossen’s equations ground the control algorithms in physical reality, allowing the software to intelligently counteract complex hydrodynamic forces using a computationally efficient 3-DOF model. Finally, Unreal Engine acts as the ultimate proving ground, offering a photorealistic and physically accurate SITL environment where perception AI and navigation nodes can be tested to the limit safely before the physical prototype is deployed.

Ultimately, this multidisciplinary architecture is not just a theoretical exercise, it is a strategic necessity. By mastering and implementing these tools, the Autonomous Systems department ensures that a vessel like the SP01 is not only safe and reliable, but also very competitive. This foundation will allow the team to continuously iterate on complex autonomous prototypes, pushing the boundaries of what sustainable marine robotics can achieve on the open water.

## References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [2] S. Macenski, A. Soragna, M. Carroll, and Z. Ge, “Impact of ros 2 node composition in robotic systems,” *IEEE Robotics and Autonomous Letters (RA-L)*, 2023.
- [3] R. Andrade, “What is ros2? - framework overview.” YouTube Video, 2021.
- [4] T. I. Fossen, “Marine craft model,” 2023.
- [5] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, 2nd ed., 2021.
- [6] EVNautilus, “Beyond the wow: The six types of ship motion.” YouTube Video, 2020.
- [7] H. NG, “Six types of ship motion explained,” 2023.
- [8] Epic Games, “Unreal engine 5.7 documentation,” 2026.
- [9] Wikipedia contributors, “Unreal engine — wikipedia, the free encyclopedia,” 2024.
- [10] U. Poseidon, “Unreal engine 5: Master boat tutorial (water series).” YouTube Video, 2024.
- [11] Husarion, “Ros 2 introduction tutorial,” 2023.
- [12] Wikipedia contributors, “Robot operating system — wikipedia, the free encyclopedia,” 2024.
- [13] Wikipedia contributors, “Six degrees of freedom — wikipedia, the free encyclopedia,” 2024.